

Lab0 实验报告

周益轩 25303090066

1 实验任务一

1.1 问题一：多人协作开发经历

在数学建模大赛中经历过多人开发（虽然项目规模比较小），具体方式如下：

1. 文档：飞书协作
2. 代码：VS Code 中的 Live Share 插件
3. 报告撰写：Overleaf 协同

1.2 问题二：为什么 Git 要分为“暂存”与“提交”两步

Git 之所以将提交过程分为“暂存”和“提交”两个步骤，核心原因是为了让开发者能够更精确地控制提交内容。

在实际开发中，开发者常常会同时修改多个文件，或者在同一个文件中进行多处不同类型的修改，而这些修改并不一定都属于同一个变更单元。如果 Git 只提供直接提交，开发者就只能选择全部提交或放弃修改，很难对提交内容进行细致调整。

暂存区的存在解决了这一问题。开发者可以先将需要提交的修改加入暂存区，确认内容无误后再执行提交，从而保证每一次提交都包含一组相关的修改，而不是把临时文件、调试代码或无关改动一起记录到项目历史中。并且暂存区快照与后续工作区编辑相互隔离，提交内容稳定可控。

同时，暂存区也起到了检查作用。在提交之前，开发者可以查看当前暂存了哪些内容，避免误提交，这对于保持项目历史的整洁和后续的版本回溯都很有帮助。

查阅资料可知，暂存区在 Git 内部被称为索引（index），本质上是一份记录着文件路径、权限模式与 blob 哈希的清单，用 `git ls-files --stage` 就能看到每一行形如 `100644 <hash> 0 < 文件名>` 的记录，行末的数字是冲突层级，正常文件为 0。正因为它是这样一份独立且已经定格的清单，`git commit` 实际做的并不是把工作区打包，而是由索引写出一棵树对象（对应底层的 `git write-tree`），再与父提交、作者信息一起封装成 commit——提交的内容取自索引，与工作区此刻的状态无关，这也解释了为什么已经暂存的改动不会因为继续编辑文件而改变。由此再看几个常用命令的分工就很清楚了：`git diff` 比较的是工作区与索引，`git diff --cached` 比较的是索引与 HEAD，而 `git add -p` 允许把同一个文件里的多个改动块分别决定是否写入索引，这正是“一次编辑拆成多个逻辑提交”得以实现的技术前提。再往深处说，Git 中实际存在 HEAD、索引、工作区三种状态，`git reset --soft / --mixed / --hard` 三者的差别，正是把 HEAD 依次同步到其中哪一层；而在合并或变基产生冲突时，Git 会把同一文件的共同祖先、本方版本与对方版本分别以 1、2、3 级写入索引，只有用 `git add` 把选定的一版提升为第 0 级，冲突才算真正解决。可见索引并不只是提交前的一个缓冲

区，它同时是提交的构造来源、版本比较的基准和冲突解决的工作台，是贯穿 Git 多个核心机制的基础结构。

1.3 问题三：git branch 与 git branch -a 的区别

git branch 与 git branch -a 的区别，在于二者列出的分支范围不同。前者只列出本地分支，也就是我们可以直接切换、提交、删除的那些；后者在本地分支之外，还把远程跟踪引用一并列出，显示为 `remotes/origin/xxx` 的形式（-a 是 --all 的缩写）。需要说明的是，远程跟踪引用并不是远程仓库里的分支本身，而是本地保存的一份“上次 fetch 时远程长什么样”的副本，因此它只是用来对照的参照，本地分支才是真正操作的对象；如果只想看远程那一半，用 `git branch -r` 即可。默认只列本地分支，是因为日常操作几乎都针对本地分支，若把一批 `origin/xxx` 混进来，`git branch -d` 这类命令反而容易误删。

查阅资料进一步看到，这两类引用在 Git 内部是分开存放的：本地分支位于 `refs/heads/` 下，远程跟踪引用位于 `refs/remotes/origin/` 下，git branch 本质上是在遍历前者，加上 -a 后遍历范围才扩大到后者（用 `git for-each-ref refs/heads refs/remotes` 也能得到同样的结果）。它们之所以能一一对应，取决于 `.git/config` 里 `remote.origin.fetch` 那一行的 refspec，默认写作 `+refs/heads/*:refs/remotes/origin/*`，作用是把远程的每个分支映射成本地的一份 `origin/` 副本，`origin/` 这个前缀便由此而来。也正因为远程跟踪引用只是只读的镜像，它刷新要靠 `git fetch` 或 `git pull`，`git branch -a` 本身并不联网；另外 -a 输出里常见的那行 `remotes/origin/HEAD -> origin/main` 是符号引用，不是真正的分支，不宜当作新分支去检出。若要查看每个本地分支的上游追踪关系和领先、落后情况，则应使用 `git branch -vv`。

2 实验任务二

依据实验指导，完成了 `main.c` 文件中的 TODO 部分并进行一次 commit，具体信息如下：

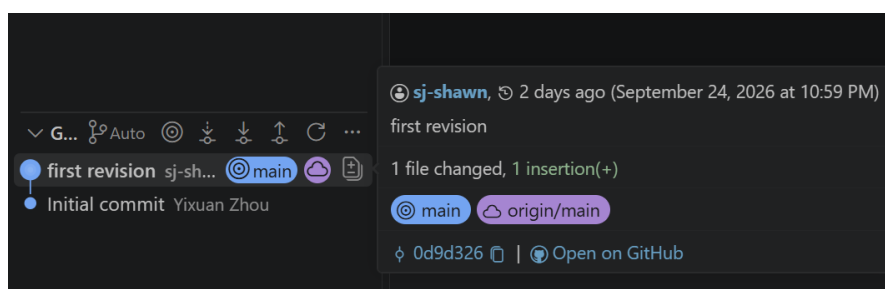


图 1: commit 记录

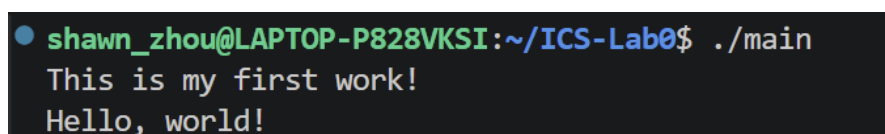


图 2: main 函数的输出

3 实验任务三：阅读文章

3.1 阅读文章一：Commit Message 规范

commit message 是每次提交时写的那句说明，它其实是写给未来的自己和队友看的。规范的目的是不是形式好看，而是让提交历史能被看懂、能被查找；做法上就是分层来写：一句话的标题概括改了什么，下面再可选地补充“为什么改”和相关的需求编号，且一次提交只讲一件事。

表 1: commit message 的三段结构

位置	是否必填	写什么
标题行	必填	用 type 说明改动类型（如 feat 新功能、fix 修 Bug、docs 文档），再用一句话概括。
正文	选填	交代“为什么这么改”，不必重复代码细节。
脚注	选填	关联的需求单号、问题编号，便于日后追溯。

3.2 阅读文章二：Git Flow 分支控制

它要解决什么问题？当多人同时在一个仓库里开发时，如果所有人都直接往同一条分支上提交，“谁改了什么”“哪些代码能上线”“线上出问题怎么紧急修”很快就会失控。Git Flow 的核心思想是：用分支的生命周期来表达开发阶段——把长期存在的分支和用完即删的临时分支区分开，让每一条分支都只承担一种明确的职责。

表 2: 五类分支及其职责

分支	生命周期	职责说明
main	长期	只存放已发布、随时可上线的稳定代码；每次合并对应一个版本 tag。
develop	长期	集成分支，日常开发的汇总线，代表“最新完成的功能”。
feature/*	临时	从 develop 拉出开发单个功能，完成后合回并删除。
release/*	临时	发布前的收尾分支：只修 Bug、定版本号、更新文档，不夹带新功能。
hotfix/*	临时	线上紧急缺陷修复，直接从 main 拉出，修完同时合回 main 与 develop。

3.3 为什么要学习 Git

一、学习感悟

说实话，在学这两篇文章之前，我几乎没有真正用过 Git：改文件就是直接覆盖保存，最多另存成“方案 1”“方案 2”。文章一让我明白，提交时那句说明并不是随手备注，而是写给未来的自己和队友看的“改动日记”——把改动讲清楚，别人查问题时就不必来问你。文章二则改变了我对“分支”的印象：原来它不是为了显得专业，而是在回答“这一刻哪份代码可以放心交出去”；Git Flow 把团队的约定画成了一张图，分支的走向其实就是协作的走向。

真正让我改观的是它背后的态度，而不是具体命令。一个人写代码时怎么提交都不会乱，可一旦

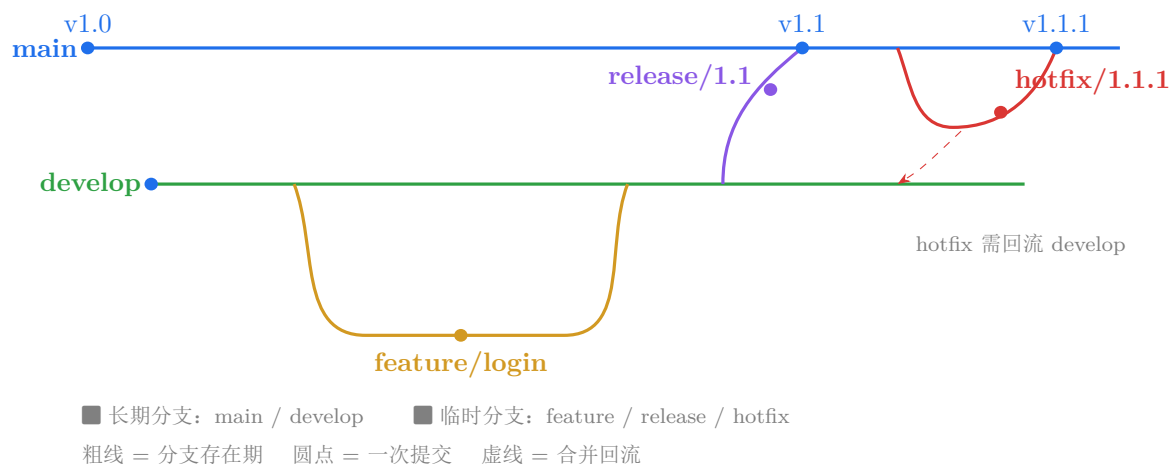


图 3: Git Flow 分支流转示意图

几个人同时改同一个仓库，就必须有一套大家都遵守的说法：谁能改、改完往哪里合并、什么可以发布。把改动说明白、把流程定清楚，本身就是对协作者负责的习惯。我现在只是刚看懂这两篇文章，还没有真正上手，但至少不再觉得 Git 神秘了。

二、分析

“Git 的价值，最终都指向同一件事：让多人协作成为可能。它看起来提供了回退、分支、提交说明等一整套功能，但这些功能其实都是为“更好地协作”而服务的。

一个人写代码时，其实用不上版本控制：改错了覆盖回去就行，也不需要向谁交代改了什么。真正需要 Git 的场景，是几个人同时改同一份代码——这时必须回答几个问题：谁改了什么、改完往哪里合并、哪份代码可以放心发布、出了问题怎么退回。Git 的回退、分支与提交记录，正是在回答这些问题。可以说，它把原本靠口头传递的信息沉淀成了明确的记录，让一群人能在同一个仓库里互不干扰地推进，也让代码的每一步变化都有据可查；而这些能力最终汇集到同一个目的——让协作变得可靠、可追溯。

随着项目变大，这些记录的价值还会继续放大：谁评审、哪次改动上线、问题出在哪一步，都能沿着提交历史查回去，这正是个人作业与真实项目之间最明显的差别之一。

4 实验任务四：分支管理与合并冲突

4.1 新建 feature 分支并分别提交

先新建 feature 分支，再分别在 feature 与 main 上修改 main.c 并各提交一次。

上图中先执行 `git branch feature` 创建分支，并用 `git branch` 确认 main 与 feature 都存在（* 表示当前所在分支）；随后运行程序，可以看到两个分支各自的修改都已生效。

4.2 合并 feature 到 main 并解决冲突

切回 main 分支执行 `git merge feature`，两次提交修改了同一行代码，因此出现了合并冲突。

如图 5，Git 把两个版本同时写进文件：<<<<<<< HEAD 与 ===== 之间是 main 的版本（`int day = 4;`），===== 与 >>>>>>> feature 之间是 feature 的版本（`int day = 1;`）。我保留了需

```
shawn_zhou@LAPTOP-P828VKSI:~/ICS-Lab0$ git branch feature
shawn_zhou@LAPTOP-P828VKSI:~/ICS-Lab0$ git branch
  feature
* main
shawn_zhou@LAPTOP-P828VKSI:~/ICS-Lab0$ make
gcc -Wall -O2 -c main.c -o main.o
gcc -Wall -O2 -o main main.o
shawn_zhou@LAPTOP-P828VKSI:~/ICS-Lab0$ ./main
3 days left until the deadline!
This is my first work!
Hello, world!
```

图 4: 新建 feature 分支并验证两次提交后的运行结果

```
C main.c 3.1 X
C main.c > main()
4 {
5     // @TODO: print a sentence you want.
6     <<<<<< HEAD      expected an expression (Current Change)
7     int day = 4;
8     ===== expected an expression
9     int day = 1;
10    >>>>>> feature    expected an expression (Incoming Change)
11    printf("%d days left until the deadline! \n", day);
12    printf("This is my first work! \n");
13    printf("Hello, world!\n");
14 }
```

图 5: 合并时出现的冲突标记

要的 `int day = 1;`，删掉冲突标记后重新 `git add` 并提交，冲突即解决。

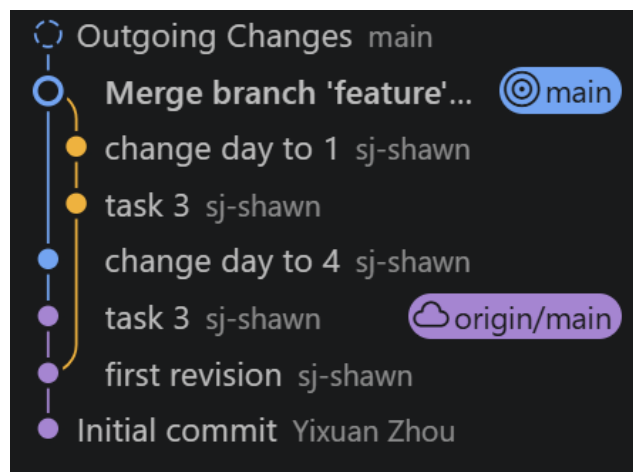


图 6: 冲突解决后 main 分支的提交历史

图 6 是解决冲突后的提交历史：合并提交“Merge branch 'feature'...”把 `feature` 上“change day to 1”的修改并入了 `main`，说明冲突已正确解决、分支已成功合并。